

- 1) The third element in an array has an index of 2 because arrays start counting at 0.
- 2) An array called quantities that holds 20 integers can be written like this: `int[] quantities = new int[20];`
- 3) An array named heights that stores 1.65, 2.15, and 4.95 is written as: `double[] heights = {1.65, 2.15, 4.95};`
- 4) A for-each loop that shows all the numbers in an array called grades looks like: `for (int grade : grades) { System.out.println(grade); }`

6) Passing a whole array to a method sends the reference for the entire array, so the method can access or change the real array. Passing one element only sends the value at that index (for primitive types), so the original array doesn't change.

7) Offset indexes are used when the program counts differently than the array does. Arrays start at 0, but people often count from 1, so offsets help match real-world counting with array positions or when using only part of an array.

8) For the code: `String name = "Elaine"; System.out.println(name.charAt(3));` The letters are E(0), l(1), a(2), i(3), n(4), e(5). The output is: i

10) A dynamic array is better when you don't know how many items you will need, like when taking input from a user until they decide to stop. A regular array can't grow on its own, but an ArrayList can.

11) The `indexOf()` method in an ArrayList checks if something is equal by using the `equals()` method. If `element.equals(object)` is true, then `indexOf()` returns that position.

12) Wrapper class values (like Integer or Double) should be compared using `equals()` and not `==`. For example: `Integer a = 5; Integer b = 5; if (a.equals(b)) { System.out.println("Equal"); }`